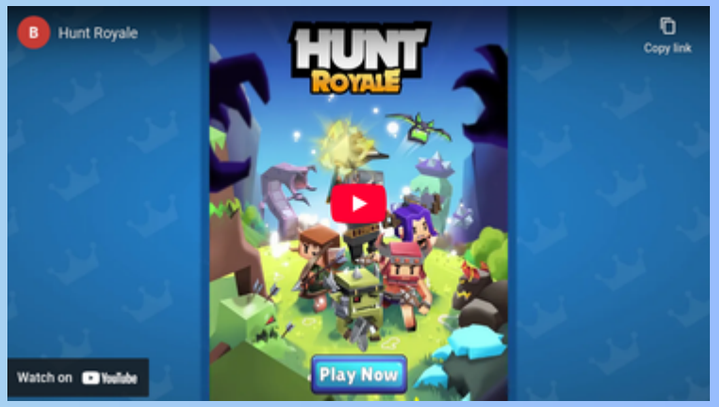
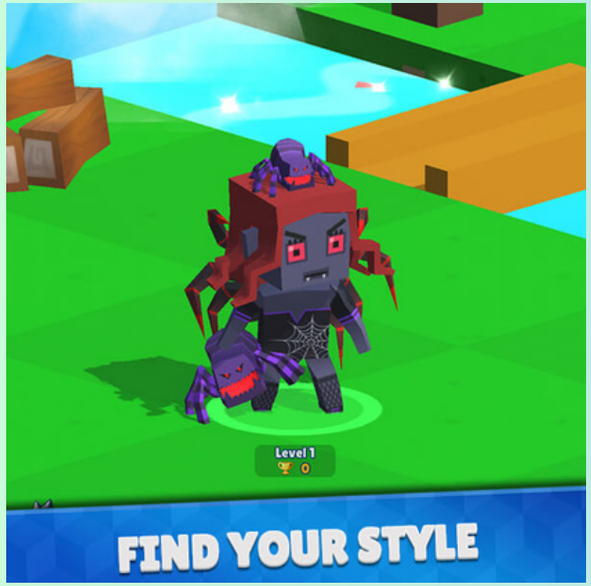
Specialization - Advanced Gameplay Programming
Hunt Royale

I chose to develop a game similar to Hunt Royale, developed by BoomBit, for my assignment because it includes a wide range of complex and engaging game mechanics. The game features player, enemy, and AI agent movement, combat systems, and behavior logic. The main objective is to compete against other players or AI-controlled opponents by collecting more points within a limited time.



https://boombit.com/games/hunt-royale-en/



To achieve this, players must hunt various creatures, some of which are stronger and provide higher rewards. The game also includes multiplayer elements, AI competitors, character progression systems, and both in-game and out-of-game upgrades.

Since it contains many interconnected systems such as combat mechanics, AI behavior, scoring systems, and progression features, I believe it is highly suitable for demonstrating technical and programming skills. Additionally, the game is very popular and engaging, which makes it an exciting and relevant choice for this assignment.

FIRAT KOCABACAK

Game Analysis

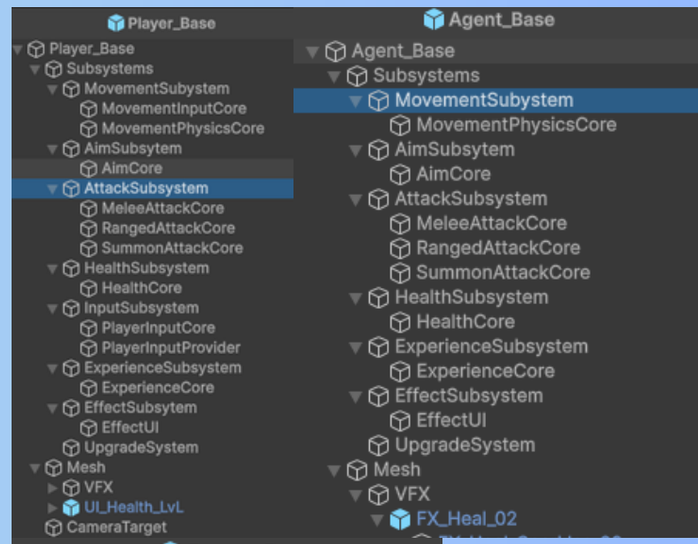
Game Structure and Core Mechanics Analysis

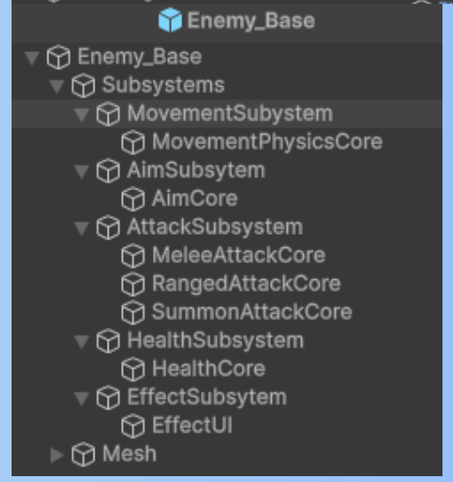
System Architecture Overview

In my implementation, I structured the project using a layered architecture composed of Core Components, Subsystems, and State Machines.

Core Components – Calculation Layer

Core Components are responsible for low-level mathematical operations and data processing.
They handle calculations such as damage formulas, movement speed evaluation, cooldown timing, and stat aggregation.
These components do not make decisions about behavior.
They simply compute and return values.
This separation ensures that mathematical logic remains reusable and independent from gameplay behavior.





Core Components
↓
Subsystems
↓
State Machine
↓
Gameplay Execution

Subsystems – Logic Layer

Subsystems are responsible for controlling how systems behave.
They use data provided by Core Components to execute gameplay logic.

For example:

- The MovementSubsystem decides how movement should be applied.
- The AttackSubsystem determines when and how an attack is executed.
- The HealthSubsystem manages damage application and death handling.

Subsystems act as the bridge between raw calculations and gameplay behavior.

FIRAT KOCABACAK

Game Analysis

Game Structure and Core Mechanics Analysis

States – Execution Layer

At the highest level, I use a State Machine architecture to control when systems are active.
For example:

- PlayerMoveState activates the MovementSubsystem.
- EnemyAttackState triggers the AttackSubsystem.
- IdleState disables certain systems.

States define when and under what conditions subsystems should operate.

```
protected override void CreateStates()
{
    IdleState = new BotIdleState(this);
    ChaseState = new BotChaseState(this);
    AttackState = new BotAttackState(this);
    HealState = new BotHealState(this);
}
```

The BaseEntity class defines a generic architecture for all entities.
Each entity initializes its own subsystems and creates its own state set.
This ensures behavioral flexibility while keeping the base structure consistent.

```
public interface IState
{
    13 başvuru
    void Enter();
    13 başvuru
    void LogicUpdate();
    13 başvuru
    void PhysicsUpdate();
    13 başvuru
    void Exit();
}
```

Each concrete entity overrides CreateStates() to define its behavioral states.
This allows different AI behaviors without modifying the core architecture.

I chose an IState interface instead of abstract base states to avoid deep inheritance and keep each state modular and independent.

```
public abstract class BaseEntity : MonoBehaviour
{
    protected Subsystem[] subsystems;

    protected HealthSubsystem healthSubsystem;
    protected EffectSubsystem effectSubsystem;

    29 başvuru
    public StateMachine SM { get; protected set; }
    11 başvuru
    public bool IsDead => healthSubsystem != null
                          && healthSubsystem.MaxHP > 0
                          && healthSubsystem.IsDead;
    5 başvuru
    public HealthSubsystem Health...
    Unity İletisi | 8 başvuru
    protected virtual void Awake()...

    3 başvuru
    protected void Initialize()...

    Unity İletisi | 6 başvuru
    protected virtual void Start()...
    Unity İletisi | 8 başvuru
    protected virtual void Update()...

    Unity İletisi | 0 başvuru
    protected virtual void FixedUpdate()...

    Unity İletisi | 2 başvuru
    protected virtual void OnDisable()...
    1 başvuru
    public virtual void ApplyEffect(StatusEffect effect,
    1 başvuru
    public virtual void OnDealDamage(int damage)...
    10 başvuru
    protected virtual void OnDied() { }
    5 başvuru
    protected abstract void CreateStates();
```

FIRAT KOCABACAK

System Communication & Event Flow

System Communication & Decoupling

Basic Signals

If systems directly reference each other, the project becomes tightly coupled and difficult to maintain.

I implemented a signal-based communication system to reduce dependencies between subsystems.

For example:

- HealthSubsystem emits death and damage signals.
- ExperienceSubsystem listens to death events.
- UpgradeSubsystem modifies stats dynamically.
- Systems do not directly depend on each other.

```
7 başvuru
public class BasicSignal<T1, T2> : BasicSignal
{
    private Action<T1, T2> m_action;


    3 başvuru
    public void Connect(Action<T1, T2> a_action)
    {
        m_action += a_action;
    }

    3 başvuru
    public void Disconnect(Action<T1, T2> a_action)
    {
        m_action -= a_action;
    }

    5 başvuru
    public void Emit(T1 a_first, T2 a_second)
    {
        m_action?.Invoke(a_first, a_second);
    }
}
```

```
/// Subscribes a method to an event.
/// </summary>
2 başvuru
public void Subscribe(string eventName, Action<object> listener)
{
    if (!_eventDictionary.ContainsKey(eventName))
    {
        _eventDictionary[eventName] = listener;
    }
    else
    {
        _eventDictionary[eventName] += listener;
    }
}

/// <summary>
/// Unsubscribes a method from an event.
/// </summary>
2 başvuru
public void Unsubscribe(string eventName, Action<object> listener)
{
    if (_eventDictionary.ContainsKey(eventName))
    {
        _eventDictionary[eventName] -= listener;
        if (_eventDictionary[eventName] == null)
        {
            _eventDictionary.Remove(eventName);
        }
    }
}
```

Systems communicate through events instead of direct references, ensuring loose coupling and scalability.

I chose an IState interface instead of abstract base states to avoid deep inheritance and keep each state modular and independent.

Event Manager

Subsystems communicate through events instead of direct references.
Systems subscribe to events and react when they are triggered.
This ensures:

- Loose coupling
- Better scalability
- Cleaner architecture

FIRAT KOCABACAK

Gameplay Execution Flow

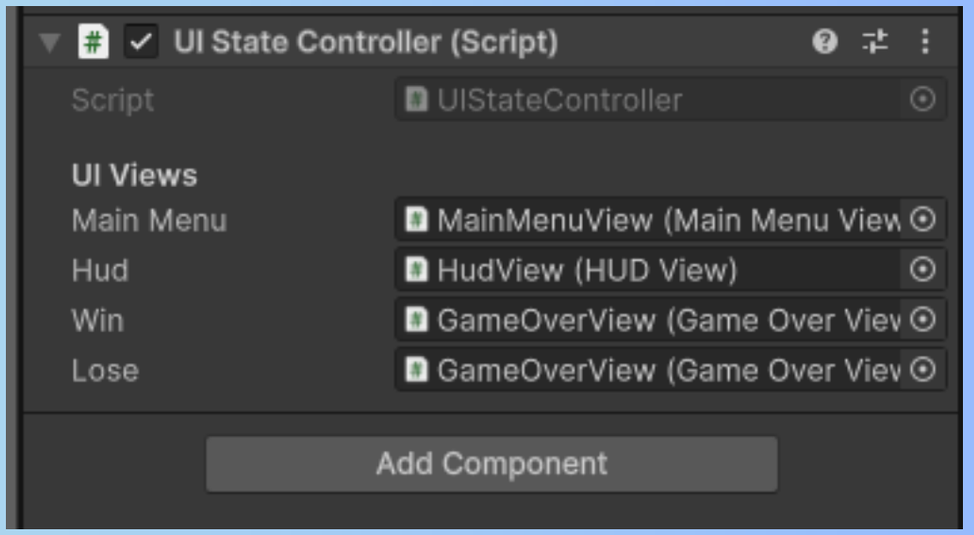
Player Input
↓
MovementSubsystem
↓
Target Detection
↓
AttackSubsystem
↓
HealthSubsystem
↓
Event Emitted
↓
ExperienceSubsystem
↓
UpgradeSubsystem

Each combat interaction triggers a chain of subsystem executions.
Every system has a single responsibility within the gameplay loop.

Game Flow

UI Management

The UI system listens to GameStateManager and automatically switches panels based on the current state (MainMenu, Playing, Paused, GameOver).
This keeps UI transitions consistent and removes state-specific UI logic from gameplay code.
UI is state-driven: GameState changes trigger UI panel transitions automatically.



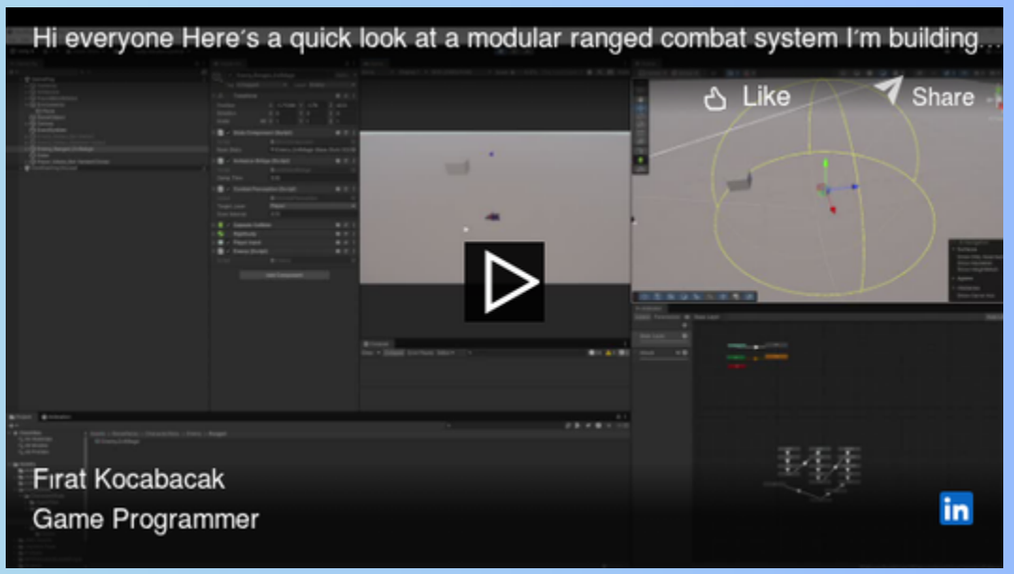
Gameplay Systems

Detection Range System

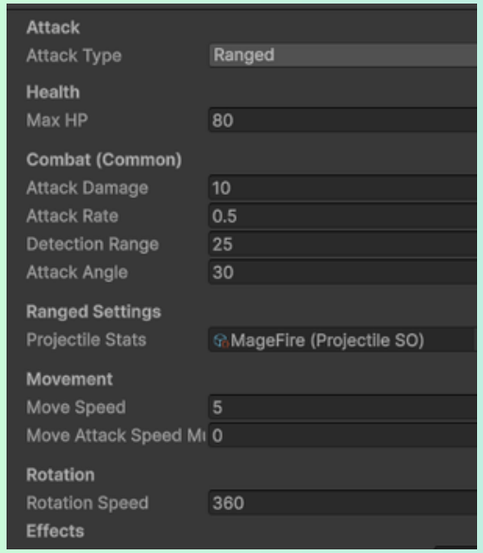
Each enemy defines its own detection behavior through a ScriptableObject-based stat configuration.
The detectionRange value is stored inside BaseStatsSO, allowing every enemy type to have a different awareness radius.

When the game runs:

- The entity periodically scans within its detectionRange.
- Valid targets are filtered by layer and distance.
- If a target enters the range, the State Machine transitions to Chase or Attack.



To Watch

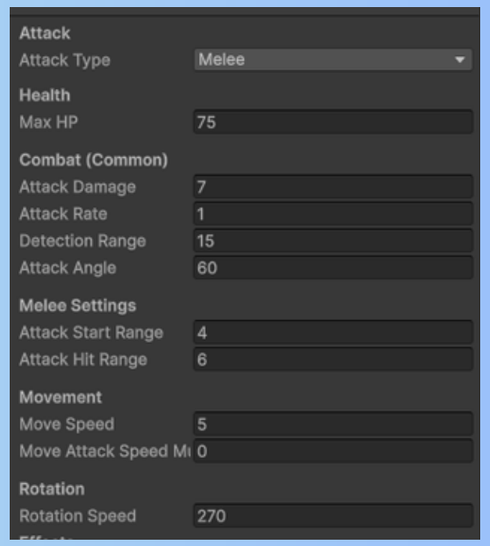


Attack	
Attack Type	Ranged
Health	
Max HP	80
Combat (Common)	
Attack Damage	10
Attack Rate	0.5
Detection Range	25
Attack Angle	30
Ranged Settings	
Projectile Stats	MageFire (Projectile SO)
Movement	
Move Speed	5
Move Attack Speed Mı	0
Rotation	
Rotation Speed	360
Effects	

This design allows:

- Per-enemy behavior customization
- Data-driven AI configuration
- Easy balancing without code changes

Detection range is fully data-driven and configurable per enemy type.



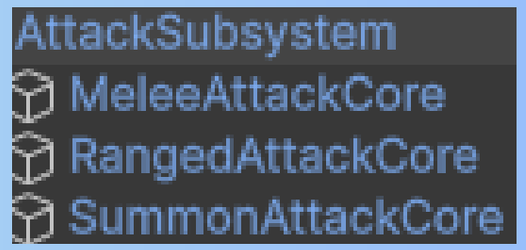
Attack	
Attack Type	Melee
Health	
Max HP	75
Combat (Common)	
Attack Damage	7
Attack Rate	1
Detection Range	15
Attack Angle	60
Melee Settings	
Attack Start Range	4
Attack Hit Range	6
Movement	
Move Speed	5
Move Attack Speed Mı	0
Rotation	
Rotation Speed	270

Gameplay Systems

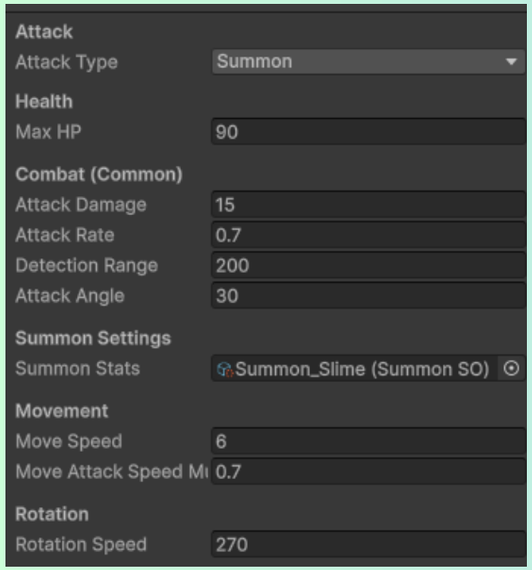
Combat System

The Combat System is built using a modular Subsystem + Core architecture.
Each entity contains an AttackSubsystem, which delegates execution to specific attack cores based on the attack type:

- MeleeAttackCore
- RangedAttackCore
- SummonAttackCore



The attack type is defined inside BaseStatsSO, making the combat behavior fully data-driven.



When a valid target is detected:

1. The State Machine transitions to AttackState
2. AttackSubsystem is activated
3. The corresponding AttackCore executes the attack logic
4. Damage is applied through HealthSubsystem

Combat behavior is determined by configuration, not hardcoded logic.

Each entity defines its attack behavior through the AttackType property stored in BaseStatsSO.
During initialization, the AttackSubsystem automatically selects the corresponding attack core:

- Melee → MeleeAttackCore
- Ranged → RangedAttackCore
- Summon → SummonAttackCore

Only the selected core is activated, while the others remain disabled.
This allows:

- Clean separation of attack logic
- Modular combat behavior
- Easy addition of new attack types
- Fully data-driven configuration

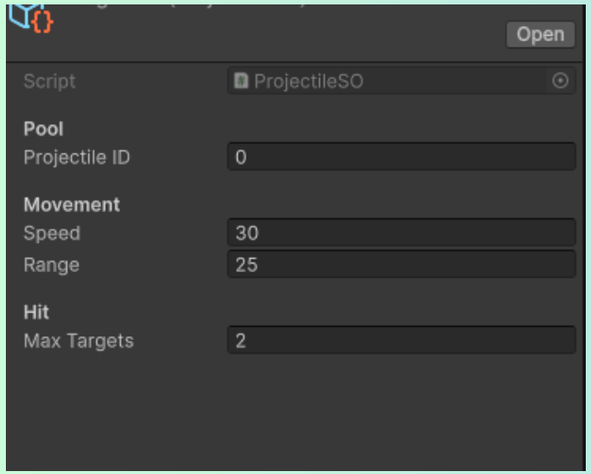
Attack behavior is selected automatically based on configuration, not hardcoded logic.

FIRAT KOCABACAK

Gameplay Systems

Data-Driven & Modular Design

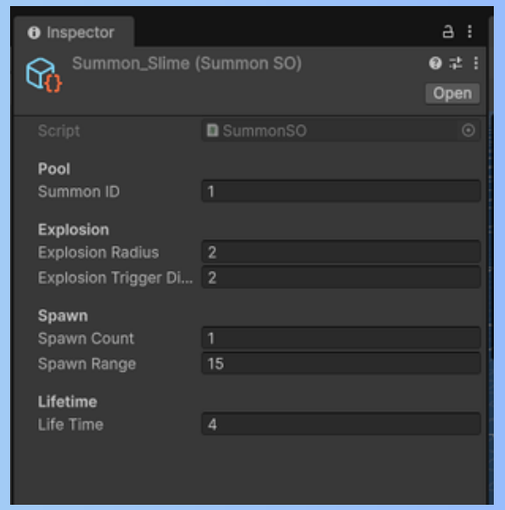
Each gameplay mechanic is implemented as a separate module (Subsystem/Core) and configured through ScriptableObjects.



This allows changing key parameters without modifying code, such as:

- Projectile pierce count
- Projectile speed and range
- Attack damage and rate
- Detection range
- Summon stats

The system is designed for fast balancing and easy feature extension.

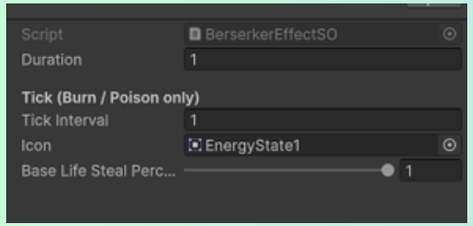


FIRAT KOCABACAK

Gameplay Systems

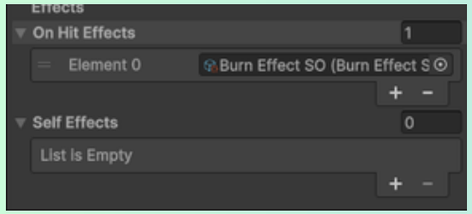
Effect System Architecture

Effects are implemented using ScriptableObjects (StatusEffectSO) and applied through the EffectSubsystem.



Each entity can have:

- OnHitEffects (applied to the target when damage is dealt)
- SelfEffects (applied to the attacker after a successful hit)



This system is fully data-driven, allowing easy tuning of:

- Duration
- Tick rate
- Effect strength

Effects are modular and reusable across different enemies and weapons.

AI Decision System

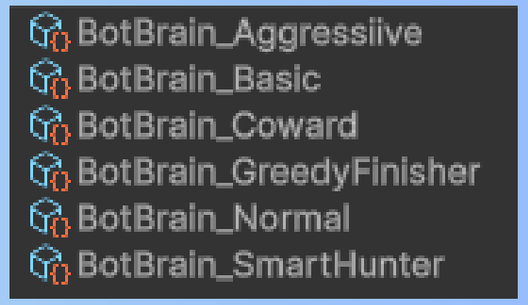
Smart Target Scoringystems

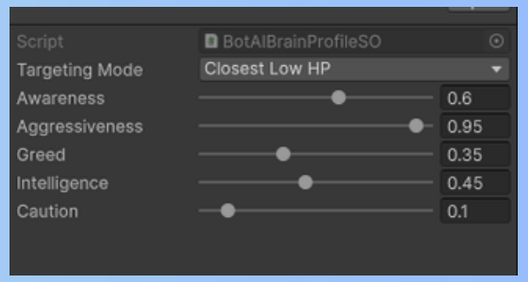
Instead of selecting targets randomly, bots evaluate enemies using a dynamic scoring system.

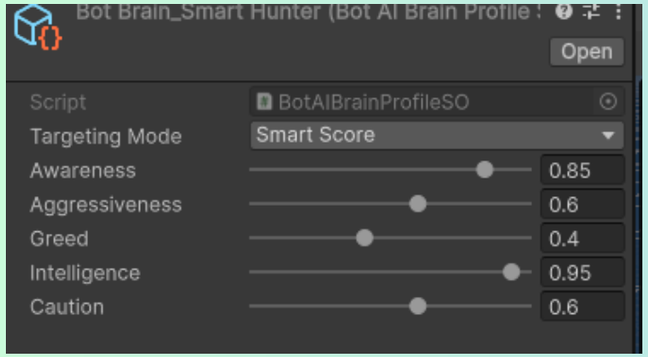
In SmartScore mode, each potential target receives a score based on:

- Distance (awareness)
- Target's remaining health (intelligence + aggressiveness)
- Risk factor (caution when low HP)
- Reward value (experience, score, star drop chance)
- Target claim count (competition penalty)

The target with the highest calculated score is selected.
Bots make strategic decisions based on weighted parameters rather than fixed rules.







When the agent's HP drops below a dynamic threshold, it switches to BotHealState and moves to the Heal

The heal threshold is not fixed; it changes based on the AI profile (aggressiveness vs caution), creating different survival personalities.

FIRAT KOCABACAK

Performance & Optimization

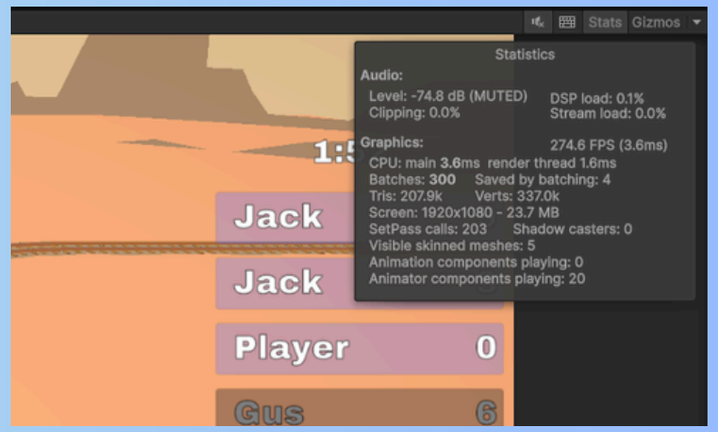
ObjectPools&Spawners

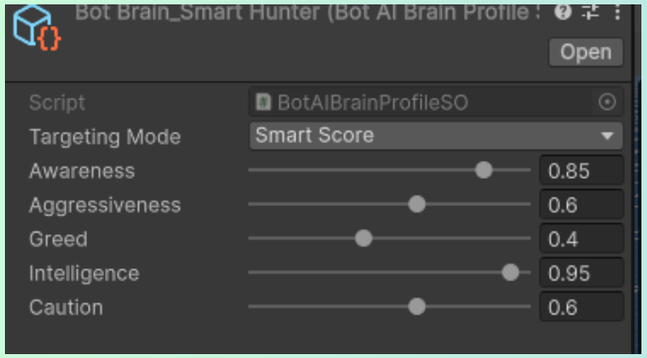
To keep runtime performance stable, I avoid frequent instantiation and expensive per-frame checks.

Optimizations used:

- Object Pooling for enemies and VFX (reused objects instead of Instantiate/Destroy)
- Preloading VFX objects at startup to prevent frame spikes
- Timed scanning (interval-based logic) instead of scanning every frame
- Squared distance checks for range comparisons

The project focuses on reducing GC allocations and minimizing expensive runtime operations.





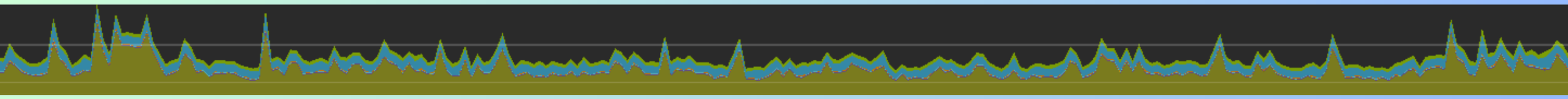
Attack Collision Optimization

Combat Hit Detection Optimization

- Melee attacks use temporary sphere triggers
- No continuous overlap checks
- Activated only during attack window
- Layer-filtered collision detection

Result:

- Minimal physics overhead
- Clean animation-synced damage system



External Assets & Tools Used

- Polygon Western Pack - Environment & stylized western assets
- Polygon Particle FX - Stylized VFX elements
- A Pathfinding Project* - AI navigation and pathfinding
- DOTween - UI transitions and smooth animations
- TextMesh Pro - Advanced text rendering
- Joystick Pack - Mobile input support
- Tony P Colors / Polyart Packs - Additional stylized assets

Technical Information

Engine Version
Unity 6 (6000.0.56f1) - DirectX 11

Platform Target
Windows,Android,WEBGL

Render Pipeline
URP

Architecture Approach
Modular, Subsystem-based, Data-Driven

Future Improvements

Technical Improvements

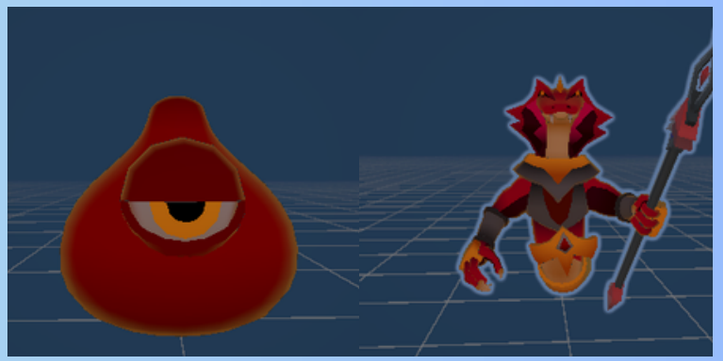
- Refactor some managers to better respect Single Responsibility Principle (SRP)
- Reduce cross-manager responsibilities and improve modular boundaries
- Improve state handling separation between AI, Combat and Navigation
- Further optimize AI scanning & scoring calculations
- Add profiling-based micro-optimizations where needed
- Improve dependency injection instead of direct singleton access in some systems

Gameplay Improvements

- New enemy archetypes with unique AI behaviors
- Additional attack patterns and boss mechanics
- Expanded Status Effect combinationsFurther optimize AI scanning & scoring calculations
- Alternative game modes (Survival / Timed / Arena variants)
- Better reward scaling & progression balancing

Visual & Presentation Improvements

- Updated character models and animation polish
- Improved VFX feedback (hit reactions, impact flashes, damage numbers)
- Better UI transitions & dynamic HUD effects
- Environment variation & lighting improvements
- Enhanced spawn and death effects



GitHub Repository

firat0667/HuntRoyale: FirstRelease

FIRAT KOCABACAK